

AXIOM v1.4 - WHEN Construct

Implementation Changelog

Generated: 2026-04-15

1 Overview

The WHEN construct adds declarative conditional concept activation to the Axiom Language System. Instead of always injecting every concept into the system prompt, WHEN rules let the .axiom file declare which concepts should activate based on properties of the incoming task. This keeps prompts lean and targeted -- only relevant behavioral overlays are applied at runtime.

Key Principle:

WHEN is an Axiom-native construct -- it expresses conditional behavior declaratively, not procedurally.

2 Files Changed

<code>axiom_files/parser.py</code>	Core DSL parser -- parsing, compilation, runtime dispatch, serialization
<code>axiom_files/validator.py</code>	Language validator -- field registration and semantic validation
<code>axiom_files/concepts.axiom</code>	Shared concept library -- new AmbiguityResolution concept
<code>axiom_files/worker.axiom</code>	Worker agent definition -- WHEN block with 3 conditional rules

3 parser.py -- Detailed Changes

3.1 Parsed Dict -- added "when" key

Added "when": [] to the parsed dictionary in `load_axiom()`. This stores the raw list of WHEN rule strings parsed from the .axiom file, parallel to existing keys like constraints, rules, and process.

```
"when": []      # <-- new key, after "concepts"
```

3.2 Section Header -- WHEN recognition

Added WHEN to the section header chain in the line parser. When the parser encounters a line equal to "WHEN", it sets `current_section = "when"`. Subsequent bullet lines (starting with "- ") are appended to `parsed["when"]` by the existing generic handler.

```
elif line == "WHEN":
    current_section = "when"
```

3.3 compile_decision_table(parsed) -> dict

New function. Compiles the WHEN block into a keyword-to-concept lookup table.

Algorithm:

1. Iterate over `parsed["when"]` entries
2. Skip entries that do not contain "activate"
3. Split on "activate" -- left side is condition, right side is concept name
4. Extract keyword from condition using markers: involves, is, contains, requires, about
5. Preserve original case of the concept name (e.g. "UncertaintyBound")
6. Return {keyword: concept_name} dict

```
Input: "if task involves uncertainty, activate UncertaintyBound"
Output: {"uncertainty": "UncertaintyBound"}
```

3.4 `apply_decision_table(task, table) -> list`

New function. Takes a task string and the compiled decision table. Returns a list of concept names whose keyword appears in the lowercased task text. Underscores in keywords are replaced with spaces before matching.

3.5 `get_prompt_with_when(agent_name, task) -> str`

New function. The primary runtime entry point for WHEN-aware prompt generation.

Pipeline:

1. `load_axiom(agent_name)`
2. `compile_decision_table(parsed)`
3. `apply_decision_table(task, table) -> activated concepts`
4. Merge shared concept library from `concepts.axiom`
5. Filter concepts:
 - If WHEN activated any concepts, keep only those
 - Otherwise, fall back to keyword-based `detect_concepts()`
6. `to_system_prompt(parsed) -> final prompt string`

Design choice: WHEN rules take priority over keyword detection. If no WHEN rule matches, the system gracefully falls back to the existing `detect_concepts()` behavior from v1.3.

3.6 `save_axiom()` -- WHEN serialization

Added WHEN block serialization after the CONCEPT blocks in `save_axiom()`. If `parsed["when"]` is non-empty, emits a blank line, the WHEN header, and each rule prefixed with "- ".

4 `validator.py` -- Detailed Changes

4.1 `_KNOWN_FIELDS` -- added "when"

Added "when" to the `_KNOWN_FIELDS` set so that the WHEN section is recognized as a valid field during Phase 1 structural validation.

4.2 Phase 3d -- WHEN entry validation

New semantic check added after 3c (constraint/rule overlap). Iterates over `parsed["when"]` entries and flags any entry that does not contain the word "activate" as an error. Every WHEN rule must specify an activation target.

Error format:

WHEN entry missing activation target: '<entry text>'

Suggestion:

Add ', activate ConceptName' to the WHEN rule.

5 `concepts.axiom` -- New Concept

Added the AmbiguityResolution concept to the shared concept library:

```
CONCEPT AmbiguityResolution
PURPOSE Detect and respond to underspecified or vague user input
APPLIES WHEN underspecified vague ambiguous unclear missing incomplete
REQUIRES Identify what information is missing before attempting an answer
EFFECT Agent must request clarification rather than guess or hallucinate
```

This concept is activated by the worker's WHEN rule: "if user input is underspecified, activate AmbiguityResolution". It enforces that the agent asks for clarification instead of guessing when input is vague.

6 worker.axiom -- WHEN Block

Added the following WHEN block after the SUCCESS section:

```
WHEN
- if task involves uncertainty, activate UncertaintyBound
- if task involves optimization, activate RewardGuard
- if user input is underspecified, activate AmbiguityResolution
```

Each rule follows the pattern: condition + marker keyword + activate + ConceptName. The compile_decision_table() function extracts the keyword after the marker (involves, is, etc.) and maps it to the concept name for runtime matching.

7 Verification Results

7.1 Validator

```
WORKER.axiom      -- STATUS: VALID
EVALUATOR.axiom   -- STATUS: VALID
REWRITER.axiom    -- STATUS: VALID
```

7.2 Runtime Tests

Input Task	Expected Behavior	Result
Task: "design a reward function"	RewardGuard activated	PASS
Task: "write a poem"	No concept activated (clean)	PASS
Task: "vague and underspecified"	AmbiguityResolution activated	PASS
Task: "estimate the uncertainty"	UncertaintyBound activated	PASS

7.3 Decision Table Compiled

```
{"uncertainty": "UncertaintyBound",
 "optimization": "RewardGuard",
 "underspecified": "AmbiguityResolution"}
```

8 Design Notes

Case preservation

compile_decision_table() preserves the original casing of concept names from the .axiom file. The condition side is lowercased for matching, but the activation target retains its PascalCase form (e.g. UncertaintyBound, not uncertaintybound).

Graceful fallback

If no WHEN rule matches the task, get_prompt_with_when() falls back to the v1.3 detect_concepts() keyword matching. This ensures backward compatibility.

No external paradigms

The WHEN block is an Axiom-native construct. It expresses conditional behavior declaratively -- no if/else branching, no loops, no external programming patterns.

Serialization round-trip

save_axiom() correctly serializes WHEN entries, and load_axiom() correctly parses them back. The round-trip is lossless.